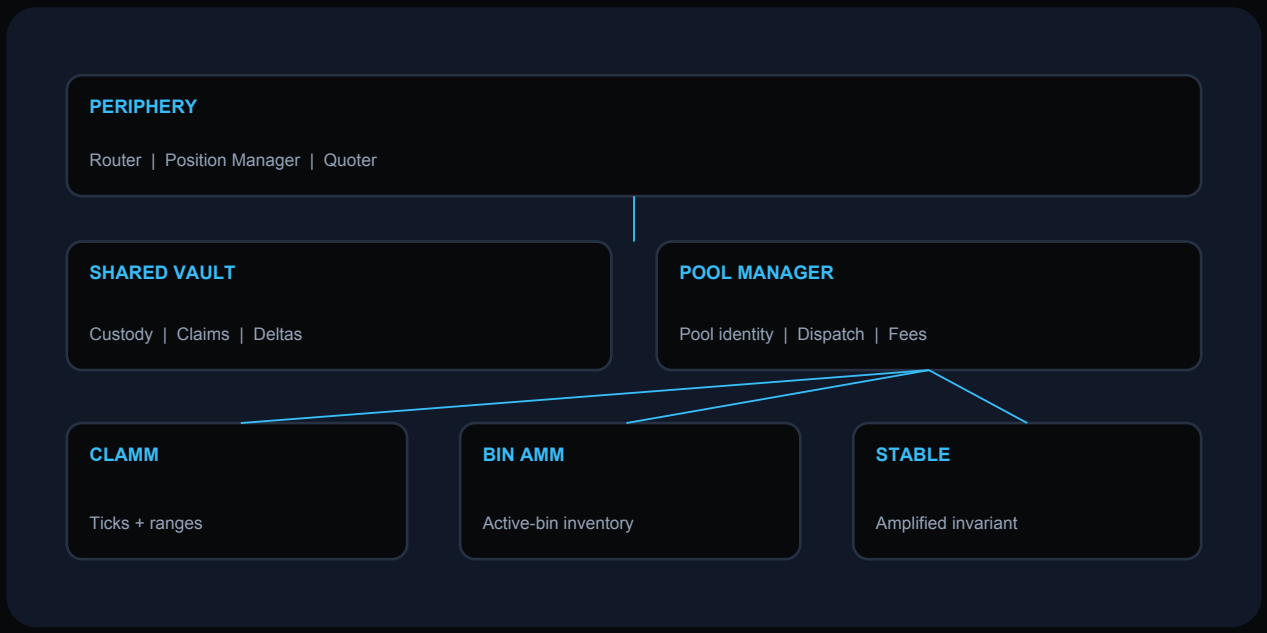
SUPADAO DEX / TECHNICAL PUBLICATION

SupaDAO DEX White Paper

A unified singleton exchange architecture for multiple liquidity regimes



VERSION 1.0.0 · 15 OCTOBER 2026 · UNDER TECHNICAL REVIEW

Repository-derived documentation. No independent audit or public deployment was confirmed.

Document control

This publication is reproducible from Markdown sources in the SupaDAO DEX repository. It describes inspected source and is not legal, tax or investment advice.

Contents

01 A Unified Singleton Exchange Architecture for Multiple Liquidity Regimes
02 Executive summary
03 1. Why multiple liquidity models?
04 2. AMMs in plain English
05 3. Liquidity mechanisms
06 4. Architecture
07 5. Swap lifecycle
08 6. Accounting and fees
09 7. Usage scenarios
10 8. Security architecture and residual risk
11 9. Testing and deployment status
12 10. Limitations
13 11. Roadmap by validation stage
14 12. Conclusion
15 References
16 Glossary
17 Revision record
18 Risk notice

A Unified Singleton Exchange Architecture for Multiple Liquidity Regimes

White paper version: 1.0.0 Publication date: 2026-10-15 Document status: Under Technical Review Scope: Repository implementation reviewed at publication time

SupaDAO DEX is independent from the SupaCore institutional settlement project. This document describes the DEX only. No independent security audit or public-network deployment was confirmed during this review.

Executive summary

SupaDAO DEX is an experimental decentralized exchange implementation that coordinates multiple automated-market-maker designs through one shared manager and custody/accounting vault. A pool selects one of three specialized engines: concentrated liquidity, discretized liquidity bins, or a two-asset amplified stable invariant. Pool identity also includes currencies, fee, spacing and plugin address, so different pools for the same token pair remain economically distinct.

“Singleton” here describes shared coordination and settlement, not a single monolithic contract and not automatic liquidity aggregation. `SupaPoolManager` validates and dispatches pool operations; `SupaVault` custodies tokens, represents internal balances as ERC-6909 claims and requires transient currency deltas to net to zero before an unlock session ends. Specialized engine contracts retain curve-specific state and mathematics.

The architecture aims to reduce duplicated exchange infrastructure while preserving liquidity models suited to different assets. It also provides plugins for dynamic fee experiments and anti-JIT residency logic. The LVR mechanism should be read as an experimental adaptive-fee design, not insurance: its internal oracle uses pool ticks rather than an external fair-price oracle, and current post-swap wiring does not update the plugin’s stored tick. Its economic effect is therefore not established.

The repository contains Solidity implementations, Foundry tests, deployment scripts, a router, quoter, position NFTs, circuit breakers and timelock administration. This is meaningful engineering evidence, but no independent audit, verified public testnet deployment, mainnet deployment or production operation was established.

1. Why multiple liquidity models?

An AMM replaces a continuously maintained order book with deterministic state transitions. Traders pay assets into a pool and receive assets out according to a curve; liquidity providers supply the inventory and generally earn a portion of swap fees. The curve decides how liquidity is distributed across possible prices.

No single distribution dominates every market. Concentrated liquidity can place capital inside selected ranges but demands range management. Discrete bins represent inventory at price levels and can offer fixed-price execution within a bin, but require rules for allocation and traversal. A stable invariant can reduce slippage near parity for economically similar assets, but becomes sensitive to depegs, decimal/rate assumptions and amplification parameters.

Deploying each model as an entirely separate exchange duplicates routing, accounting, permissions and integrations. A shared architecture may reduce that architectural fragmentation. It does not itself eliminate market fragmentation: separate `PoolKey` values remain separate pools, and the present quoter does not prove a universal optimizer that searches every route.

2. AMMs in plain English

Imagine a pool holding assets X and Y. A trader adds X and removes Y. The pricing rule makes Y progressively more expensive as its reserve falls. Fees compensate active liquidity providers but do not guarantee profit: asset prices may change, the pool may be adversely selected by better-informed traders, and a position can become one-sided or inactive.

For a constant-product reference model, reserves satisfy `x * y = k`. The spot price is related to the reserve ratio. SupaDAO's CLAMM does not simply apply that equation to all deposited assets; it uses Q64.96 square-root prices and liquidity active between ticks. Its stable and bin engines use different calculations.

3. Liquidity mechanisms

3.1 Concentrated liquidity

Providers choose lower and upper ticks. Liquidity contributes to swaps only while price lies in that range. For active liquidity `L` and square-root prices, token deltas follow the familiar concentrated-liquidity forms:

```
amount0 = L * (sqrt(Pb) - sqrt(P)) / (sqrt(P) * sqrt(Pb))
```

```
amount1 = L * (sqrt(P) - sqrt(Pa))
```

where `Pa` and `Pb` bound the position and `P` is current price. The Solidity implementation uses Q64.96 fixed-point arithmetic, explicit rounding helpers, initialized-tick bitmaps and fee growth inside ranges. Out-of-range positions stop earning swap fees until price returns, and may hold mostly one asset.

3.2 Discretized bins

`BinAMMEngine` derives a bin identifier from the initial tick and spacing. Each bin stores two reserves, total liquidity and fee growth. A swap in the active bin uses `BinMathLib` to convert at that bin's price, capped by available output reserves. This supports fixed-price execution inside the bin, but the current engine does not walk to neighboring bins after exhausting active-bin inventory. It should not be described as globally zero-slippage.

3.3 Stable invariant

`StableAMMEngine` stores two reserves and a default amplification parameter. It iterates toward an invariant `D` and solves a new opposing reserve `Y` after input. Amplification makes the curve flatter near balance than constant product, which may reduce slippage for assets that reliably track the same unit of value. The implementation is two-asset and does not normalize token decimals or consume rate oracles; pool creators and integrators must enforce compatible units. A depeg can rapidly convert the pool into the weaker asset.

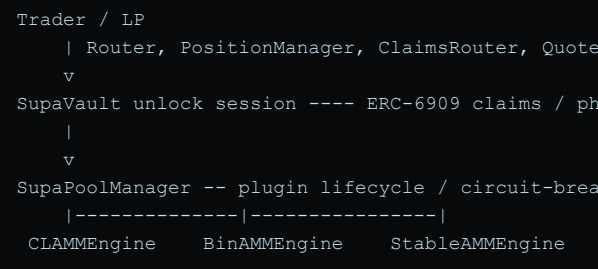
3.4 Loss-versus-rebalancing

LVR measures a form of adverse selection: when an AMM price is stale relative to a better-informed market, arbitrage can update the pool while extracting value that a continuously rebalanced benchmark would retain. It differs from impermanent loss, which compares holding a pool position with holding the original assets, and from a smart-contract exploit.

Fees may offset some adverse selection but also raise trader costs. `LVRShieldPlugin` computes an override fee from a base fee, prior fee, elapsed time and an internal tick-deviation estimate, then clamps it between configured bounds. This is an endogenous signal, not an external truth oracle. Moreover, the stored tick supplied during post-swap oracle writes is not refreshed from current pool state in the reviewed code. Claims of measured

or guaranteed LVR reduction are therefore unsupported.

4. Architecture



`PoolKey(currency0, currency1, fee, tickSpacing, plugin, curveType)` is hashed into `PoolId`. The manager holds canonical initialization, price/liquidity summaries and protocol-fee accrual; engines hold detailed curve state. The vault authorizes pool managers to post account deltas. At the end of each unlock callback, every currency delta associated with the caller must be settled, taken, minted as claims or burned from claims so the nonzero count returns to zero.

Plugins are optional pool-level addresses. The dispatcher reads a permission bitmask and invokes selected lifecycle callbacks. A separately configured whitelist can restrict plugins at initialization. External calls enlarge the trust and denial-of-service surface, so the whitelist, hook gas behavior and plugin code are part of each pool's risk profile.

5. Swap lifecycle

1. A trader sends an exact-input or exact-output request to `SupaRouter`, including a deadline, path and slippage bound.

2. The router opens a vault unlock session. The vault records the router as locker in transient storage.

3. The router decodes the path and calls `SupaPoolManager.swap` for each hop.

4. The manager derives `PoolId`, checks pause state and initialization, then asks the plugin for any fee override.

5. The selected curve engine computes amount deltas, changes curve state and accrues LP/protocol fees.

6. The manager runs the after-swap plugin callback, posts token deltas to the vault and synchronizes price/liquidity summaries.

7. The router pays inputs and takes outputs using tokens, native currency or claims according to the call path.

8. The vault verifies that no transient currency delta remains; otherwise the entire transaction reverts.

Different engines share the envelope but not the invariant. That distinction is central: unified accounting does not make CLAMM ranges, bins and the amplified stable curve mathematically interchangeable.

6. Accounting and fees

Pool fees use hundredths of a basis point (`1e-6`), so `3000` represents 0.30%. Swap code separates a configured protocol share and adds the LP portion to curve-specific fee-growth accumulators. PositionManager "pokes" a position with zero liquidity change to synchronize fees, then represents amounts owed as ERC-6909 claims before collection.

Protocol fees accrue by currency in the manager. Only the configured controller can collect them. `SupaFeeCollector` can distribute collected values to DAO treasury, LP staking and insurance reserve addresses according to owner-set basis points. These are configurable destinations, not evidence of a token, staking program or approved economic policy.

7. Usage scenarios

Trader. A user selects a route and supplies a minimum output. The relevant engine prices the trade; plugin fees may change the effective fee. The transaction reverts if the deadline or output bound fails. Risks include price movement, MEV, insufficient liquidity, hook failure and asset behavior.

Concentrated LP. A provider chooses ticks and deposits required token amounts. Capital can be efficient within range, but inactivity and one-sided exposure occur outside it. Fees depend on actual trade flow and active liquidity.

Bin LP. A provider currently interacts with active-bin accounting. Available reserves cap the output; adjacent-bin traversal is not present in the reviewed swap path.

Stable-pool LP. Two assumed-equivalent assets are deposited into an amplified pool. Near-balance execution may be flatter, but decimal mismatch, rate drift and depeg risk are material because no normalization/rate-oracle layer was found.

All numerical outcomes presented by a simulator are illustrative unless a dated benchmark and methodology accompany them.

8. Security architecture and residual risk

The vault’s callback lock and transient zero-delta check protect the settlement boundary. Authorized managers alone can post third-party deltas. Routers enforce deadlines and slippage limits. Solidity 0.8.26 checked arithmetic is used except where libraries justify unchecked regions. Circuit breakers can pause vault, curve or pool scopes; guardians pause and the owner unpauses. A custom timelock can delay administration. Only the vault is UUPS upgradeable in current source.

Residual risks include implementation errors in custom math, rounding at boundaries, malicious/non-standard tokens, hook reentrancy or griefing, governance or upgrade-key compromise, parameter abuse, oracle manipulation, liquidity manipulation, MEV, depegs and denial of service. Tests and formal-style properties improve confidence but do not replace an independent audit, public bug bounty, deployment rehearsal or operational controls.

9. Testing and deployment status

Foundry is configured for Solidity 0.8.26, Cancun, IR compilation, optimizer runs of 1,000,000, fuzz runs of 1,024 and invariant campaigns of 256 runs by depth 64. The repository contains unit, fuzz, invariant, integration, benchmark, formal-style and deployment-script sources. On 15 October 2026, the default `forge test --summary` command executed 17 tests across 6 suites: all 17 passed, with no failures or skips. This is the current reproducible result, not evidence that every test source in the tree executed. A simultaneous `forge snapshot --check` failed because measured gas diverged from the checked-in snapshot and several tests had no snapshot entry. Historical larger totals therefore require a documented command/configuration before they can be used as current proof.

Deployment scripts exist for the contract suite and local test tokens. No canonical public-chain address registry, explorer verification record or independently corroborated public testnet/mainnet deployment was found. A hosted frontend, if reachable, is not proof that protocol contracts are publicly deployed.

10. Limitations

• The LVR plugin’s tick/oracle wiring limits the intended adaptive signal.

• The bin engine does not traverse adjacent bins in the reviewed swap function.

• The stable engine lacks decimal/rate normalization and supports two assets.

• A universal pair-wide route optimizer was not found.

• TVL valuation is not a canonical on-chain protocol primitive.

• Plugin/oracle write permissions and configuration require additional threat review.

• Only the vault implements UUPS upgradeability; administrative centralization persists until a verified handoff.

• No independent audit or public deployment was confirmed.

11. Roadmap by validation stage

Completed in repository: core contracts, three engine types, periphery, claims, governance controls, local deployment scripts and broad Foundry suites.

Required before public testnet: resolve documented engine/plugin limitations, reproduce full tests and gas snapshots, establish canonical addresses, publish deployment configuration and conduct targeted adversarial review.

Required before mainnet readiness assessment: independent audit, remediation and retest; public testnet observation; token-compatibility policy; operational key/guardian procedures; monitoring, incident response and a public bug bounty.

12. Conclusion

SupaDAO DEX’s proposition is architectural: share custody, settlement and lifecycle infrastructure while selecting curve logic per pool. That can simplify integrations and make different liquidity regimes available through a common envelope, but it concentrates correctness and governance requirements in shared components. The next credible milestone is not a broader performance claim; it is resolving known limitations and validating the system independently on a public testnet.

References

1. Milionis, Moallemi, Roughgarden, Zhang, “Automated Market Making and Loss-Versus-Rebalancing,” 2022/2024, https://arxiv.org/abs/2208.06046

2. Egorov, “StableSwap - efficient mechanism for Stablecoin liquidity,” 2019, https://docs.curve.finance/assets/pdf/whitepaper_stableswap.pdf

3. Uniswap Labs, v4 concepts and PoolManager documentation, https://docs.uniswap.org/contracts/v4/concepts

4. Solidity documentation, https://docs.soliditylang.org/en/v0.8.26/

5. Foundry invariant testing documentation, https://getfoundry.sh/forge/invariant-testing

6. EIP-1153, Transient storage opcodes, https://eips.ethereum.org/EIPS/eip-1153

7. EIP-6909, Minimal multi-token interface, https://eips.ethereum.org/EIPS/eip-6909

Glossary

AMM: on-chain mechanism that quotes trades from deterministic state. CLAMM: AMM with liquidity assigned to price ranges. Invariant: mathematical relationship the swap transition preserves or solves. LVR: adverse-selection loss relative to continuous rebalancing. MEV: value extractable through transaction inclusion or ordering. Singleton: shared coordination/accounting instance for many pools. Slippage: difference between expected and executed price as size/state changes. TVL: value assigned to deposited assets, requiring a valuation methodology. Liquidity bin: inventory assigned to a discrete price level. StableSwap: amplified invariant family intended for correlated-value assets.

Revision record

| Version | Date | Status | Change |

| --- | --- | --- | --- |

| 1.0.0 | 2026-10-15 | Under Technical Review | Initial evidence-based publication set. |

Risk notice

This document is technical information, not legal, tax or investment advice. Decentralized exchange use can result in loss through smart-contract defects, market movement, adverse selection, liquidity conditions, asset failure, execution and governance actions.